title: “Prácticas de Git y GitHub” subtitle: “Módulo PI2 · ASIR2” author: “Paco Aldarias” date: “30 de octubre de 2026” lang: es-ES

Realiza las actividades en un repositorio propio para la unidad. No utilices datos reales ni guardes contraseñas, tokens, claves privadas u otra información confidencial. Conserva el mismo repositorio para las prácticas siguientes.

Cuenta de GitHub Education y primer archivo

Objetivo

Crear una cuenta de GitHub, solicitar la verificación de estudiante de GitHub Education y dejar un archivo de texto con tu nombre en un repositorio.

Pasos

1. Crea una cuenta personal en GitHub o inicia sesión con la cuenta que ya tengas.
2. Visita GitHub Education y solicita la verificación como estudiante. Sigue las instrucciones que aparezcan para acreditar tu situación académica. La aprobación depende de los criterios actuales de GitHub y puede tardar; no es necesario publicar documentación académica en un repositorio.
3. Mientras se tramita la solicitud, crea en GitHub un repositorio para estas prácticas llamado `ud5-practica`. Elige si será público o privado según las indicaciones del profesor. Si lo haces público, no añadas datos personales adicionales ni información sensible. El profesor prefiere sea público.
4. En el repositorio, crea un archivo de texto cuyo nombre incluya tu nombre y apellidos, en minúsculas y sin espacios ni tildes. Por ejemplo, `pacoaldarias.txt`.
5. Escribe dentro del archivo tu nombre y apellidos, sin añadir contraseñas, correo personal, dirección ni otros datos. Guarda el cambio con el mensaje `docs: añadir archivo personal de la práctica`.
6. Comprueba en GitHub que el archivo aparece en la rama `main`. Cuando se verifique tu condición de estudiante, revisa los beneficios disponibles en GitHub Education y activa Copilot si te corresponde.
7. Clona el repositorio en tu equipo para continuar las actividades:

```
git clone https://github.com/TU-USUARIO/ud5-practica.git
cd ud5-practica
git status
```

Sustituye `TU-USUARIO` por tu nombre de usuario de GitHub. Si el repositorio es privado, inicia sesión con el método de autenticación configurado para GitHub.

Entrega

- Enlace al repositorio de GitHub.
- Captura o evidencia de que el archivo `nombreapellido.txt` está en `main`.
- Estado de la solicitud de GitHub Education. No incluyas capturas con datos académicos personales o información sensible.

Crear una rama y preparar una pull request

En tu copia local del repositorio:

1. Actualiza `main` y crea una rama para documentar un procedimiento:

   ```
   git switch main
   git pull --ff-only
   git switch -c docs/procedimiento-backup
   ```

2. Crea `procedimiento-backup.md` con los pasos para comprobar una copia de seguridad de laboratorio. Incluye cómo verificar el resultado y evita escribir credenciales o rutas con datos sensibles.

3. Comprueba los cambios y crea un commit:

   ```
   git status
   git diff
   git add procedimiento-backup.md
   git commit -m "docs: añadir procedimiento de backup"
   ```

4. Publica la rama y abre una pull request desde `docs/procedimiento-backup` hacia `main`:

```
git push -u origin docs/procedimiento-backup
```

Si falla crear un token y ponerlo a la url de repo remoto asi:

git remote set-url origin https://<TU_TOKEN>@github.com/tu-usuario/tu-repositorio.git

5. Describe el objetivo de la PR, resume los cambios y añade una lista de comprobaciones. Revisa los cambios antes de fusionar la PR.

Entrega

- Enlace a la PR.
- Mensaje de commit claro.
- Procedimiento revisado y sin datos confidenciales.

Simular y resolver un conflicto

Haz esta actividad después de integrar la PR anterior. Trabaja únicamente en este repositorio de prácticas.

1. Actualiza `main` y crea la primera rama:

```
git switch main
git pull --ff-only
git switch -c conflicto-a
```

2. Crea `configuracion-laboratorio.txt` con una línea inicial, por ejemplo: `servidor_dns=192.0.2.10`. Guarda el archivo en un commit.

3. Crea otra rama desde `main` que cambie esa misma línea:

```
git add configuracion-laboratorio.txt
git commit -m "docs: documentar DNS del laboratorio"
git switch main
git switch -c conflicto-b
```

4. En `conflicto-b`, crea el mismo archivo con un valor de ejemplo distinto, como `servidor_dns=192.0.2.53`, y confirma el cambio.

5. Fusiona `conflicto-a` en `main`; después intenta fusionar `conflicto-b`:

```
git switch main
git merge conflicto-a
git merge conflicto-b
git status
```

6. Resuelve el conflicto editando el archivo y dejando una configuración coherente. Elimina las marcas `<<<<<<<`, `=======` y `>>>>>>>`. Comprueba el contenido y finaliza el merge:

```
git diff
git add configuracion-laboratorio.txt
git commit
git status
```

Entrega

- Archivo final sin marcas de conflicto.
- Historial de commits con la integración.
- Explicación breve de cómo decidiste el valor que conservaste.

Guardar cambios con stash

1. En `main`, crea o modifica un archivo de notas y deja un cambio sin confirmar.
2. Guarda el cambio en el *stash* con un mensaje y comprueba la lista:

```
git stash push -m "notas de laboratorio pendientes"
git stash list
git status
```

3. Consulta el contenido guardado y recupéralo sin eliminar la entrada:

```
git stash show -p stash@{0}
git stash apply stash@{0}
```

```
git diff
```

4. Cuando hayas verificado que recuperaste el trabajo correcto, elimina esa entrada:

```
git stash drop stash@{0}
git stash list
```

Entrega

Indica qué diferencia hay entre `git stash apply` y `git stash pop` y en qué momento eliminaste la entrada.

Comparar reset soft y mixed

Realiza esta actividad en una rama temporal y con cambios de prueba. No uses `--hard`.

1. Crea una rama y dos commits de prueba:

```
git switch main
git switch -c prueba-reset
```

   Crea y confirma un archivo de texto sencillo; después modifica una línea y crea un segundo commit. Anota el resultado de `git log --oneline -3` y `git status`.

2. Deshaz el último commit conservando los cambios preparados:

```
git reset --soft HEAD~1
git status
git diff --staged
```

3. Vuelve a confirmar el cambio y deshaz el commit con la opción mixta:

```
git commit -m "test: recuperar cambio de prueba"
git reset HEAD~1
```

```
git status
git diff
```

4. Compara los resultados. El modo `--soft` deja el cambio en el área de preparación; el modo predeterminado (`--mixed`) lo conserva en el archivo, pero lo quita del área de preparación.
5. Deja activa la rama `prueba-reset` para revisar los resultados y comprueba `git status`. No cambies de rama ni la elimines mientras haya cambios sin confirmar. Decide qué hacer con los archivos de prueba antes de limpiar el repositorio.

Entrega

Describe qué cambió en la referencia de la rama, el área de preparación y los archivos de trabajo en cada modo.